

고급 파이썬 프로그래밍 및 자동화 최종 보고서

연구 코드 최적화: 자료구조 · 제너레이터 · 클래스 설계 · 데코레이터 적용

학번 / 이름: 202655251 / 조주민

0. 개요

본 보고서는 멀티홉 질의응답 연구에 사용되는 코드(**Forward-Looking Guidance, 이하 FG**)를 대상으로, 수업에서 다룬 파이썬 자료구조·반복자/제너레이터·클래스 설계·데코레이터 기법을 적용하여 구조와 성능을 개선한 결과를 정리한다. 전체 파이프라인은 vLLM 기반 Qwen3-8B(또는 OpenAI API)와 Contriever+FAISS 검색기로 동작하나, 본 개선은 모델과 검색 결과 자체는 변경하지 않고 **매 실행·평가마다 반복 호출되는 호스트 측 파이썬 코드**만을 대상으로 한다. 따라서 정답(EM/F1) 값은 변하지 않으며, 최적화 전후 함수의 출력이 동일 한지를 무작위 입력 20,000건으로 확인하였다(불일치 0건, results/equivalence.json).

개선 항목은 네 가지이다. (A) 정답 정규화의 문자 제거를 `str.translate`로 변경하고 정규식을 모듈 로드 시 1회만 컴파일하였다. (D) 동일 정규화 함수에 `lru_cache`를 적용하였다. (B) 코퍼스 통계와 같이 전체 적 재가 불필요한 경로를 JSONL 스트리밍 제너레이터로 전환하였다. (C) OpenAI·vLLM 두 백엔드가 `if self.is_openai` 분기로 혼재하던 LLMEngine을 설정 `dataclass` + 백엔드 전략 + 레지스트리 구조로 분리하였다. 모든 수치는 동일 CPU 환경에서 측정한 값이며, 본문·표·그림의 값은 모두 `benchmark_results.csv`에서 산출된다.

1. 연구 코드 소개

FG는 계획 기반 반복 검색(iterative RAG)으로 멀티홉 질문에 답하는 기법이다. 플래너가 Q1, Q2, ... 형태의 하위 질문 계획을 생성하고, 실행기가 각 단계를 해결할 때 **아직 해결되지 않은 하위 질문**을 추출기(Extractor)와 추론기(Reasoner)에 함께 제공한다. 이로써 중간 산출물이 이후 단계의 재사용을 고려하여 생성된다. 본 기법은 STRIDE, DualRAG, Self-Ask 세 베이스라인에 플러그인 형태로 적용되며, 2WikiMultiHopQA·HotpotQA·MuSiQue에서 EM과 token-F1로 평가된다.

1.1 대상 코드 선정

개선 대상으로는 핵심 알고리즘 파일(`methods/stride.py` 등)이 아니라 `common.py`의 평가·입출력·문자열 유틸리티와 LLMEngine을 선정하였다. 해당 코드는 **방법 × 데이터셋 × {baseline, +FG} 조합을 실행할 때 마다 가장 빈번히 호출되는 공통 코드**이기 때문이다. 한 데이터셋은 질문 1,000개로 구성되고 정답에는 별칭(alias)이 포함되며 결과를 hop별로 재집계하므로, `normalize_answer` 등의 함수는 한 번의 평가에서 수만 회 호출된다. 또한 LLMEngine은 로컬 vLLM과 OpenAI API를 한 클래스가 함께 처리하여, 백엔드 추가나 실험 설정 변경 시 반복적인 수정이 요구되는 구조이다.

입출력 측면에서 대상 코드의 입력은 (i) 모델 예측과 정답 문자열 쌍, (ii) 질문·문맥이 담긴 JSONL 파일, (iii) 백엔드 선택용 모델 id이며, 출력은 EM/F1 점수, 코퍼스 통계, 생성 텍스트이다. 모두 모델 외부의 일반 파이썬 처리이므로 GPU 없이 측정 가능하다는 점도 선정에 고려되었다.

2. 기존 코드의 문제점 분석

최적화에 앞서 성능·구조상의 문제 지점을 분석하였다. 프로파일링에는 표준 라이브러리 `cProfile`과 본 과제에서 구현한 `timed` 데코레이터를, 메모리 측정에는 `tracemalloc`을 사용하였다.

2.1 반복 호출마다 재생성되는 객체 (`normalize_answer`)

정답 정규화 경로는 평가에서 가장 빈번히 호출되며, 내부의 `remove_punc`는 호출 시마다 `set(string.punctuation)`를 새로 생성하고 파이썬 수준의 제너레이터로 각 문자에 대해 집합 멤버십을 검사한다. 문장부호 32개 집합을 매 호출 재생성하는 비용도 존재하나, 더 근본적인 문제는 문자 필터가 C가 아닌 파이썬 루프로 수행된다는 점이다. `remove_articles` 역시 호출마다 동일한 정규식 두 개를 재적용한

다(파이썬이 내부 캐시를 제공하나 패턴을 사전 컴파일하는 편이 명확하다).

```
def remove_punc(text):
    exclude = set(string.punctuation) # 호출마다 재생성
    return ''.join(ch for ch in text if ch not in exclude) # 파이썬 루프
```

2.2 동일 문자열의 반복 정규화

정답에는 다수의 별칭이 존재하고, 예/아니오 질문이 많아 "yes"/"no" 등 짧은 정답이 반복된다. 또한 `evaluate.py`의 `score_by_hop`은 hop 버킷별로 한 번, 전체로 한 번 점수를 산출하여 동일 행을 두 번 정규화한다. 결과적으로 동일 문자열에 대한 `normalize_answer` 중복 호출이 상당하다.

2.3 불필요한 전체 적재 입출력

`read_jsonl`은 파일 전체를 리스트로 적재한다. `prepare_data.py`의 코퍼스 통계 산출과 같이 행을 한 번 순회하고 폐기하면 되는 작업에서도 파일 전체가 메모리에 적재된다. 질문 1,000개 규모에서는 부담이 작으나, 문맥을 펼친 후보 패시지 코퍼스나 대규모 덤프에서는 피크 메모리가 불필요하게 증가한다.

2.4 LLMEngine의 책임 혼재

LLMEngine은 로컬 vLLM 엔진과 OpenAI API라는 성격이 다른 두 백엔드를 한 클래스에 두고, `__init__.chat.complete` 전 메서드에서 `if self.is_openai`로 분기한다. 샘플링 파라미터(`temperature`, `top_p`, `max_tokens`, `stop`, `seed` 등)도 각 메서드에서 그때그때 구성한다. 이로 인해 (1) 백엔드를 추가하려면 세 메서드를 모두 수정해야 하고, (2) 설정과 런타임 상태가 혼재하여 실험 조건 변경이 번거롭다. 단일책임원칙에 위배되는 구조이다.

원본 `common.py`에는 `is_openai` 분기가 4곳 존재하며, 새 백엔드 추가 시 수정해야 하는 메서드는 3개였다(5절에서 정량 비교).

3. 적용한 수업 개념

적용한 수업 개념을 먼저 표로 정리한다. 각 항목이 위 문제를 해결하는 방식은 4절에서 코드와 함께 기술한다.

분류	수업 개념	적용 위치	노린 효과
A. 자료구조/복잡도	<code>str.maketrans/translate</code> , 정규식 사전 컴파일	<code>normalize_answer</code> , <code>docs_to_str</code>	문자 처리의 지배 연산을 C 레벨로, 객체 재생성 제거
B. 반복자/지연평가	<code>generator</code> , <code>lazy fold</code>	<code>iter_jsonl</code> (<code>read_jsonl</code> 분리)	전체 적재 대신 한 행씩 → 피크 메모리 절감
C. 클래스 설계	<code>dataclass(frozen, slots)</code> , <code>Protocol</code> , <code>Registry/Factory</code> , 단일책임원칙	LLMEngine → <code>GenerationConfig</code> + <code>Backend</code> + <code>registry</code>	설정/상태 분리, 백엔드 교체 용이
D. 데코레이터	<code>functools.lru_cache</code> , 파라미터화 데코레이터, <code>functools.wraps</code>	<code>normalize_answer</code> , <code>timed</code> , <code>retry</code>	중복 계산 제거, 부가 로직 분리

표와 같이 자료구조(A), 제너레이터/지연평가(B), 클래스 설계(C), 데코레이터(D)의 네 범주를 적용하였다. 과제 기준(세 가지 이상)을 충족하며, 특히 동일한 `normalize_answer` 함수가 A(`translate`)와 D(`cache`)를 함께 적용받으므로 두 기법의 기여를 분리하여 측정하였다(5.1절).

4. 개선 과정

4.1 (A) 정답 정규화: 집합·제너레이터 → str.translate + 사전 컴파일

문장부호 제거의 지배 연산은 문자열 길이에 비례하는 문자 스캔이다. 따라서 해당 스캔을 파이썬 루프가 아닌 C 수준에서 수행하는 것이 핵심이다. str.maketrans로 삭제 테이블을 모듈 로드 시 1회 생성하고 str.translate로 일괄 제거하며, 정규식도 모듈 수준에서 사전 컴파일한다. 출력은 기존과 완전히 동일하다.

```
# after
_PUNCT_TABLE = str.maketrans('', '', string.punctuation) # 한 번만
_QUOTE_RE = re.compile(r"'|\"|'|\\|'")
_ARTICLE_RE = re.compile(r"\b(a|an|the)\b")

def remove_punc(text):
    return text.translate(_PUNCT_TABLE) # C 레벨 단일 패스
```

이 변경만으로 정규화는 입력 크기와 무관하게 일정 비율로 단축된다(측정값은 5절). deque나 heapq 등 컨테이너 교체형 최적화는 본 코드에 해당하지 않는다. 이 함수의 병목은 컨테이너 선택이 아니라 문자 단위 처리의 실행 수준(파이썬 대 C)에 있기 때문이다.

4.2 (D) 같은 정규화에 lru_cache

2.2의 중복을 제거하기 위해 메모이제이션을 적용한다. 입력이 문자열이고 출력이 결정적이므로 functools.lru_cache를 그대로 사용할 수 있다.

```
@functools.lru_cache(maxsize=1 << 18)
def normalize_answer(s):
    return white_space_fix(remove_articles(remove_punc(lower(s))))
```

캐시 이득은 정답 반복도에 비례하며, 입력이 모두 고유하면 거의 없다. 이에 5절에서 반복도가 낮은 경우와 높은 경우를 구분하여 A(translate)와 D(cache)의 기여를 분리하였다. maxsize는 2^18로 설정하였으며, 이는 한 평가의 고유 정답 수를 충분히 포괄하면서 메모리 상한을 두기 위함이다(한계는 6절).

4.3 (B) JSONL 스트리밍 제너레이터

read_jsonl을 유지하되, 한 행씩 산출하는 iter_jsonl을 별도로 추가하였다. 리스트 인덱싱이 실제로 필요한 소수의 호출부는 read_jsonl을 사용하되 그 구현을 list(iter_jsonl(...))로 변경하여 중복을 제거하였다. 코퍼스 통계와 같이 한 번 순회하여 접는(fold) 작업은 제너레이터로 처리하면 파일 전체가 적재되지 않는다.

```
def iter_jsonl(path):
    # 한 행씩 yield
    with open(path, encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if line:
                yield json.loads(line)

def read_jsonl(path):
    # 인덱싱이 필요한 곳만
    return list(iter_jsonl(path))
```

4.4 (C) LLMEngine 분해: 설정 dataclass + 백엔드 전략 + 레지스트리

책임을 세 가지로 분리하였다. 첫째, 샘플링 파라미터는 불변 GenerationConfig dataclass로 묶었다(frozen=True, slots=True). 둘째, 두 백엔드를 각각 단일 책임의 OpenAIBackend·VLLMBackend로 분리하고, 공통 규약은 Backend Protocol로 정의하였다(상속 없는 구조적 타이핑). 셋째, 모델 id에 따른 백엔드 선택은 레지스트리에 위임하였다. LLMEngine은 기존 시그니처를 유지하는 파사드로 두어 호출부 변경이 불필요하다.

```

@dataclass(frozen=True, slots=True)
class GenerationConfig:
    temperature: float = 1.0; max_tokens: int = 512; top_p: float = 0.95
    think: bool = False; stop: tuple | None = None; seed: int | None = None

class Backend(Protocol):
    def chat(self, messages_list, cfg: GenerationConfig) -> list[str]: ...
    def complete(self, prompts, cfg: GenerationConfig) -> list[str]: ...

BACKEND_REGISTRY = [
    (lambda mp: mp.startswith('gpt-') or mp.startswith('openai/'), OpenAIBackend),
    (lambda mp: True, VLLMBackend),          # 기본: 로컬
]

def build_backend(model_path, **kw):
    for matches, factory in BACKEND_REGISTRY:
        if matches(model_path):
            return factory(model_path, **kw)

```

백엔드를 추가할 경우 Backend Protocol을 만족하는 클래스를 작성하고 레지스트리에 매칭 함수 한 줄을 추가하면 되며, chat·complete 등 기존 메서드는 수정하지 않는다. 무거운 의존성(vllm, openai, transformers)은 각 백엔드의 __init__ 내부에서만 import하도록 지연시켰다.

4.5 (D) timing/retry 데코레이터

부가 로직 또한 본체에서 분리하였다. timed는 누적 실행시간을 함수 속성에 기록하여 프로파일링에 사용하고, retry는 불안정한 OpenAI API 호출을 지수 백오프로 재시도한다(파라미터화 데코레이터). 두 데코레이터 모두 functools.wraps로 원본 메타데이터를 보존하며, 재시도가 모두 실패하면 원래 예외를 그대로 재전파한다.

단, retry는 원본 _openai_chat에 없던 새 정책이다. 따라서 앞서 기술한 출력 동일성은 예외가 발생하지 않는 성공 경로에 한하며, 일시적 실패 시 지연시간과 실제 API 호출 횟수는 원본과 달라진다(최종 실패 시 원래 예외를 재전파하므로 결과의 의미는 보존된다).

```

def retry(times=3, base_delay=0.5, backoff=2.0, exceptions=(Exception,)):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **k):
            delay = base_delay
            for attempt in range(1, times + 1):
                try: return fn(*a, **k)
                except exceptions:
                    if attempt == times: raise # 마지막엔 원본 예외 그대로
                    time.sleep(delay); delay *= backoff
            return wrapper
        return deco

```

4.6 고려했지만 적용하지 않은 대안

다음 항목은 검토 후 제외하였다. (1) FAISS 검색 결과 조립 루프의 numpy 벡터화는 top-k가 3~5로 작아 실익이 적고 GPU 의존이 발생하여 측정 재현이 어려우므로 보류하였다. (2) normalize_answer를 단일 정규식으로 통합하는 방안은 출력이 미세하게 달라질 위험이 있어, 동작 보존을 위해 변환 단계 순서를 유지하고 각 단계만 최적화하였다. (3) 캐시의 디스크 영구화는 단일 평가 수명 내에서는 과도하여 제외하였다.

5. 최적화 결과

모든 측정은 7절에 기재한 CPU 환경에서 수행하였다. 시간 항목은 위밍업 2회 후 11회 반복한 평균($\pm$ 표준편차)이며, 메모리는 tracemalloc 피크의 최댓값이다. GPU는 사용하지 않았다. 각 측정의 표준편차는 results/benchmark_results.csv에 모두 기록되어 있으며, 시간 측정의 변동계수(표준편차/평균)는 중앙값 약 1.2%, 전체의 96%가 10% 이내로 측정이 안정적이었다.

5.1 (A·D) 정답 정규화

translate만 적용한 버전(캐시 없음)과 원본을 비교하면 입력 크기 2,000부터 128,000까지 전 구간에서 약 **1.60배** 일정하게 단축되었다. 캐시를 추가하면 정답 반복도에 따라 달라져, 반복이 적은 경우 약 **2.8배**, 반복이 많은 경우 약 **20배**까지 단축되었다(원본 대비, $n=128,000$).

n (정규화 호출 수)	원본(ms)	translate(ms)	translate+cache(ms)	배율(캐시,저반복)	배율(캐시,고반복)
2,000	7.3	4.6	2.7	2.7x	21x
8,000	29.0	17.8	11.1	2.6x	22x
32,000	115.7	71.6	44.2	2.6x	22x
128,000	462.7	290.6	166.8	2.8x	20x

표 1. 정답 정규화 시간(저반복 기준 ms)과 캐시 배율. translate는 크기와 무관하게 일정 배율, 캐시는 반복도에 비례.

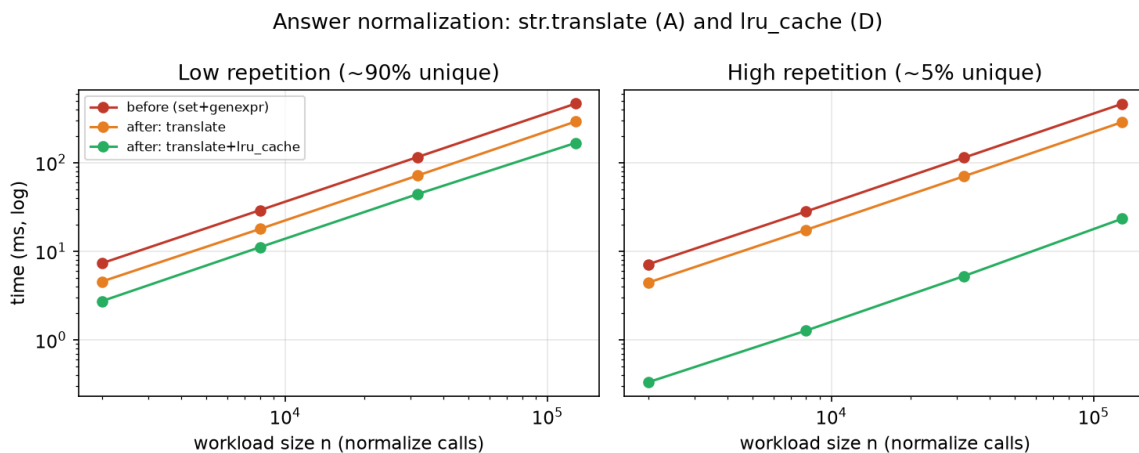


그림 1. 정규화 시간 대 입력 크기(로그-로그). 왼쪽 저반복, 오른쪽 고반복. 세 곡선의 간격이 translate(주황)와 cache(초록)의 기여를 나타낸다.

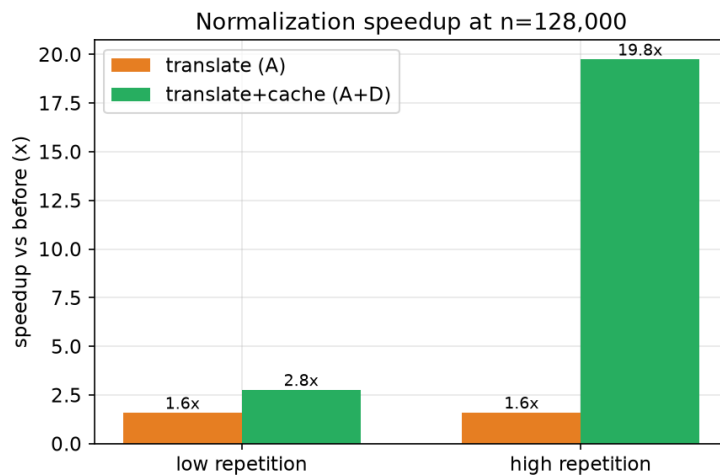


그림 2. $n=128,000$ 에서의 원본 대비 배율. 캐시 이득이 반복도에 따라 크게 달라진다.

5.2 (B) JSONL 스트리밍

코퍼스 통계(고유 패시지 수·supporting 수·hop 분포) 산출이라는 동일 작업을 전체 적재 방식과 스트리밍 방식으로 측정하였다. 두 방식의 결과 통계가 동일함은 assert로 확인하였다. 행 수가 증가해도 스트리밍의 피크 메모리는 거의 일정한 반면 전체 적재는 선형 증가하여, 최대 80,000행에서 피크 메모리가 약 **3.3배** 차이를 보였다. 또한 리스트를 생성하지 않으므로 실행 시간도 약 **30%** 단축되었다.

행 수	피크 메모리(MB) 전체 적재	피크 메모리(MB) 스트리밍	메모리 절감	시간(ms) 전체	시간(ms) 스트리밍
5,000	18.1	5.5	3.3x	46	36
20,000	72.7	22.0	3.3x	219	163
80,000	291.1	88.0	3.3x	1056	734

표 2. 코퍼스 통계 작업의 피크 메모리·시간. 스트리밍은 메모리가 거의 일정.

Streaming JSONL (B): memory flat vs growing

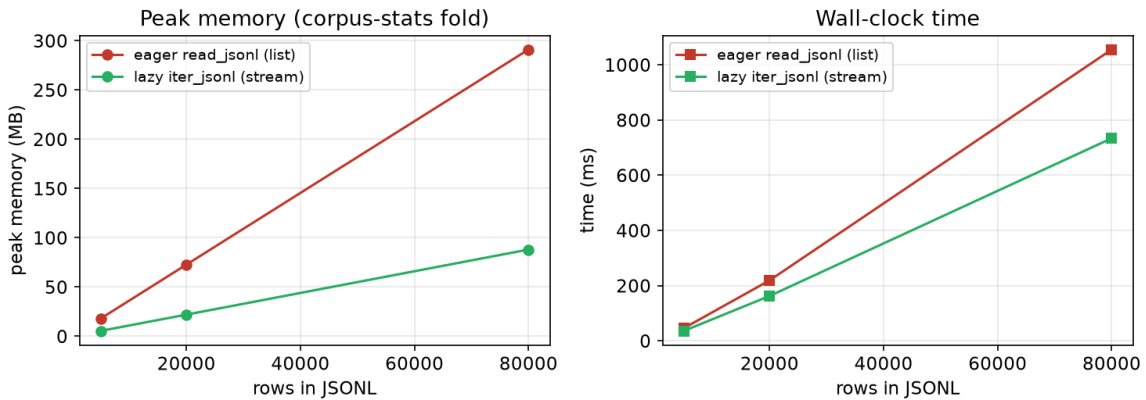


그림 3. 왼쪽 피크 메모리(전체 적재는 선형 증가, 스트리밍은 평탄), 오른쪽 실행 시간.

5.3 (C) LLMEngine 구조와 디스패치 오버헤드

구조 지표는 다음과 같다. 런타임 백엔드 분기(is_openai)가 4곳에서 1곳으로 감소하였고, 새 백엔드 추가 시 수정해야 하는 기존 메서드 수가 3개에서 0개로 줄었다(클래스 추가 + 레지스트리 한 줄). 리팩터링에 따른 런타임 비용은 모의 백엔드로 측정하였으며, 파사드+dataclass+전략 디스패치의 추가 비용은 호출당 수 마이크로초 수준이다(배치 1에서 약 4.4us 대 3.1us). 실제 호출은 LLM 생성이 초 단위로 지배하므로 해당 차이는 무시 가능하다.

지표	before	after
is_openai 런타임 분기 수	4	1
새 백엔드 추가 시 수정 메서드 수	3	0
디스패치 시간(us, batch=1, 모의)	3.09	4.44
디스패치 시간(us, batch=8, 모의)	3.40	4.46
디스패치 시간(us, batch=64, 모의)	8.48	9.53

표 3. 엔진 구조 지표와 디스패치 마이크로 벤치. (is_openai 분기 수는 소스에서 자동 산출, 수정 메서드 수는 변경 범위를 수작업 산정.) 구조는 단순해지고 런타임 비용은 무시 가능.

Engine refactor (C): structure vs micro-overhead

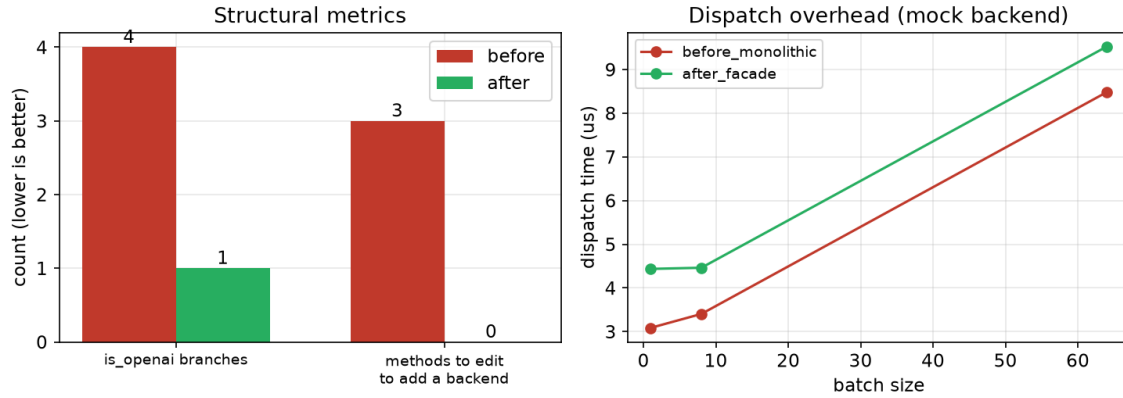


그림 4. 왼쪽 구조 지표(낮을수록 좋음), 오른쪽 모의 백엔드 디스패치 오버헤드.

5.4 (A, 부가) docs_to_str: 측정 결과 개선 없음

검색 결과를 한 문자열로 결합하는 docs_to_str의 `s += ...`를 join으로 변경하였으나, 측정 결과 **유의미한 속도 개선은 없었다**. 최대 $n=12,800$ 에서도 두 방식이 거의 동일하였다(오히려 join이 약 2% 느림). 원인은 6.4절에서 분석한다. 그럼에도 join 형태를 유지한 것은 누적 문자열 빌드의 권장 관용구이며 최악의 경우 ($O(n^2)$)를 구조적으로 회피하기 때문이다.

passages n	before s+= (ms)	after join (ms)	비율
50	0.011	0.012	0.95x
200	0.032	0.035	0.92x
800	0.122	0.129	0.95x
3,200	0.493	0.512	0.96x
12,800	2.150	2.197	0.98x

표 4. docs_to_str: CPython에서는 두 방식이 사실상 동률(비율 1.0x 근처).

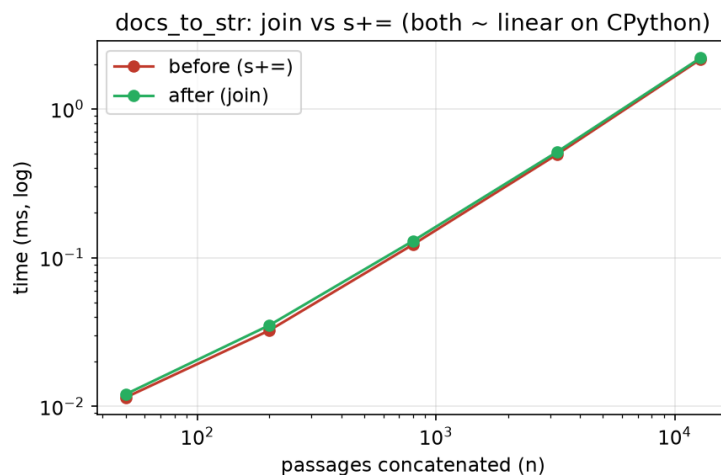


그림 5. docs_to_str: join과 s+= 모두 입력 크기에 대해 거의 같은 기울기(로그-로그). 교과서적 $O(n^2)$ 가 나타나지 않는 이유는 6.4절에서 해석한다.

6. 결과 해석 및 한계

6.1 translate가 일정 배율로 단축되는 이유

translate의 약 1.60배 단축이 입력 크기와 무관하게 일정한 것은 두 방식의 접근 복잡도가 $O(\text{문자 수})$ 로 동일하기 때문이다. 변화한 것은 상수 항이다. 원본은 문자마다 파이썬 바이트코드로 집합 멤버십을 검사하나, translate는 동일 스캔을 C 루프에서 일괄 처리한다. 따라서 비례 상수만 감소하고 기울기는 유지되어 로그-로그 그래프에서 평행선으로 나타난다. 집합을 매 호출 재생성하던 비용이 제거된 효과도 포함된다.

6.2 캐시 이득과 반복도의 관계

캐시는 적중 시 $O(1)$ 이고 미적중 시 계산과 저장이 더해져 원본보다 다소 더 소요된다. 따라서 전체 이득은 적중률에 좌우된다. 반복이 많은 평가(예/아니오 정답, 별칭, hop별 재집계)에서는 적중률이 높아 최대 20배까지 단축되었으나, 거의 고유한 입력에서는 2.8배에 그쳤다. 즉 캐시는 데이터 특성에 의존하는 최적화이다. 본 평가 파이프라인은 반복이 상당하여 이득이 있으나, 정답이 모두 고유한 작업에서는 메모리만 추가로 소비하고 이득이 미미할 수 있다.

6.3 적용 범위와 상한

B의 스트리밍은 피크 메모리를 감소시키나, 이는 전체를 동시에 보유할 필요가 없는 작업에 한정된다. 정렬이나 다회 임의 접근이 필요한 호출부는 리스트가 적합하므로 `read_jsonl`을 유지하였다. 캐시 또한 $\text{maxsize}=2^{18}$ 의 상한을 두어 메모리의 무한 증가를 방지하였으며, 이는 한 데이터셋의 고유 정답 수를 상회하는 값이다. 무한 캐시는 장기 실행 프로세스에서 메모리 누수처럼 작용할 수 있어 배제하였다.

6.4 docs_to_str의 개선 부재 원인

CPython은 참조 카운트가 1인 문자열에 대한 += 연산을 새 객체 생성 없이 제자리 확장으로 처리하는 최적화를 제공한다. docs_to_str의 s는 루프 내 유일 참조이므로 이 최적화가 적용되어 이론적 $O(n^2)$ 가 실제로 나타나지 않았다. 오히려 join은 제너레이터를 한 번 더 순회하는 비용으로 근소하게 느렸다. 다만 이 최적화는 CPython 구현과 참조 상황에 의존하므로(PyPy 실행 또는 누적 문자열이 외부에서 참조되는 경우 $O(n^2)$ 가 드러난다), 측정상 이득이 없더라도 join 관용구를 유지하는 것이 안전하다. 측정 없이 성능 개선을 주장하지 않는다는 과제 취지에도 부합한다.

6.5 한계 및 향후 과제

본 개선은 호스트 측 CPU 코드에 한정된다. 파이프라인 전체의 벽시계 시간은 LLM 생성과 검색이 지배하므로, 평가·입출력 유틸리티의 단축이 end-to-end 성능에 미치는 영향은 제한적이다. 다만 해당 영역은 수업 주제(자료구조·제너레이터·클래스·데코레이터)가 직접 적용되며 반복 실행되는 공통 코드이므로 개선의 가치가 있다. 향후 과제로는 (1) 검색 결과 조립의 배치 벡터화를 통한 GPU 경로 측정, (2) timed 데코레이터를 이용한 실제 평가 1회 프로파일링으로 호스트 측 시간 비중을 정량화하여 최적화의 유효 범위를 규정하는 작업을 고려한다.

7. 측정 환경

항목	값
OS	Linux-6.8.0-110-generic-x86_64-with-glibc2.39
Python	3.13.13 (CPython)
CPU	AMD EPYC 7413 24-Core Processor (논리 코어 96)

GPU	사용 안 함
주요 라이브러리	numpy 2.4.6, matplotlib 3.11.0
반복 측정	시간 11회(워밍업 2), 메모리 3회
난수 seed	20260621
동작 보존 검증 before/after	출력 동일성: 무작위 입력 20,000건, 총 60,500회 비교에서 불일치 0건 (results/equivalence.json)
데코레이터 의미 검증	retry 복구=True, 원예외 재발생=True, wraps 이름보존=True

8. 재현 방법 및 저장소 구조

저장소는 최적화 전후 코드를 병치하고, 벤치마크와 보고서 생성을 일괄 실행할 수 있도록 구성하였다.

```
project/
├── README.md
├── requirements.txt
├── src/before/      # 원본 코드
├── src/after/       # 최적화 코드 (공개 API 동일)
├── benchmark/run_benchmark.py  # 측정 -> results/benchmark_results.csv
├── benchmark/make_figures.py  # CSV -> results/figures/*.png
├── results/         # csv, env.json, figures
└── report/build_report.py  # 이 PDF 생성
```

실행 순서는 `pip install -r requirements.txt` 후 `python benchmark/run_benchmark.py` → `python benchmark/make_figures.py` → `python report/build_report.py`이다. 모든 과정은 CPU에서 수 분 내에 완료되며 외부 네트워크나 모델 가중치가 필요하지 않다. 최적화 전후 비교는 `src/before`와 `src/after`의 `common.py`를 비교(diff)하여 확인할 수 있다.